

Code Assessment of the Diamond PAU Smart Contracts

March 17, 2026

Produced for



by



Contents

1	Executive Summary	3
2	Assessment Overview	5
3	Limitations and use of report	16
4	Terminology	17
5	Open Findings	18
6	Resolved Findings	22
7	Informational	24
8	Notes	30

1 Executive Summary

Dear all,

Thank you for trusting us to help Sky with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Diamond PAU according to [Scope](#) to support you in forming an opinion on their security risks.

Sky designs Diamond PAU (Parallelized Allocation Units), a modular architecture using the [EIP-2535 Diamond](#) proxy pattern. It is designed to replace the legacy ALM Controller with a faceted approach where controller actions are deployed as separate contracts and can be shared with different ALM Controllers. The current implementation achieves librarification of the existing ALM Controller to facilitate a future Diamond upgrade.

The most critical subjects covered in our audit are functional correctness & equivalence, precision of arithmetic operations, and access control. Precision of arithmetic operations can be improved, see [CurveLib: Rounding Error Amplification](#).

The general subjects covered are code consistency, and migration considerations. See [Inconsistency](#).

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	8
• Code Partially Corrected	2
• Risk Accepted	5
• Acknowledged	1

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

This review is focused on the functionality equivalence of the refactoring that merges and librarificates the [Spark ALM Controller](#) and [Grove ALM Controller](#).

While most of the integrations are reviewed in separate reports, see [Spark ALM Controller v1.7.0 Review](#), [v1.10.0 Diff Review](#) and [Grove ALM Controller Review v1.8.0](#), the integrations added between [Spark v1.7.0](#) and [v1.9.0](#), namely the OTCSwap, UniswapV4, and WSTETH integrations, are assumed to be correct and secure.

The following files are in scope of this review:

```
src/
  ALMProxy.sol
  ALMProxyFreezable.sol
  ForeignController.sol
  MainnetController.sol
  OTCBuffer.sol
  RateLimitHelpers.sol
  RateLimits.sol
  WEETHModule.sol
  interfaces/
    Common.sol
    IALMProxy.sol
    ILayerZero.sol
    IRateLimits.sol
    UniswapV4.sol
  libraries/
    AaveLib.sol
    ApproveLib.sol
    CCTPLib.sol
    CentrifugeLib.sol
    CurveLib.sol
    DAIUSDSLlib.sol
    ERC4626Lib.sol
    ERC7540Lib.sol
    FarmLib.sol
    LayerZeroLib.sol
    MapleLib.sol
    MerklLib.sol
    OTCLib.sol
    PSM3Lib.sol
    PSMLib.sol
    PendleLib.sol
    SparkVaultLib.sol
    SuperstateLib.sol
    TransferAssetLib.sol
    USDELib.sol
```

```

    USDSLlib.sol
    UniswapV3Lib.sol
    UniswapV4Lib.sol
    WEETHLib.sol
    WSTETHLib.sol
    WrapProxyETHLib.sol
    uniswap-v3/
        UniswapV3OracleLib.sol
        UniswapV3UtilsLib.sol
deploy/
    ControllerDeploy.sol
    ControllerInstance.sol
    ForeignControllerInit.sol
    MainnetControllerInit.sol

```

The assessment was performed on the source code files inside the Diamond PAU repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	04 Mar 2026	6b0958f575d6fc99311df5326353f3202fba95b5	Initial Version
2	11 Mar 2026	220dbcfd6ebbf9e23f684aa9c29c8d76e14d447c	After Intermediate Report
3	13 Mar 2026	697a8dbb2cd5b46ae7b5c618e1d9c7efb112a83e	v1.11.0

For the solidity smart contracts, the compiler version 0.8.25 was chosen.

2.1.1 Excluded from scope

Any other files are out of scope.

The following are explicitly out of scope:

- Most integrations are covered in separate reviews, see Spark ALM Controller [v1.7.0 Review](#), [v1.10.0 Diff Review](#) and Grove ALM Controller Review [v1.8.0](#), the integrations added between Spark [v1.7.0](#) and [v1.9.0](#).
- The OTCSwap, UniswapV4, and WSTETH integrations are assumed to be correct and secure.
- All 3rd-party protocols that the ALMProxy integrates with are out of scope. They are assumed to work honestly and correctly as documented.
- Dependencies and libraries e.g. OpenZeppelin.

2.2 System Overview

This system overview describes the latest received version (**Version 3**) of the contracts as defined in the [Assessment Overview](#).

At the end of this section, we describe the changes introduced in each version.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Sky designs Diamond PAU (Parallelized Allocation Units), a modular architecture using the [EIP-2535 Diamond](#) proxy pattern. It is designed to replace the legacy ALM Controller with a faceted approach where controller actions are deployed as separate contracts and can be shared with different ALM Controllers. In the latest version, it librarificated the existing ALM Controller to facilitate a future Diamond upgrade.

2.2.1 ALM Controllers

ALM Controllers are a set of on-chain components of the Sky Liquidity Layers designed to manage and control the flow of liquidity on Ethereum mainnet and L2s, and between them, by leveraging DSS Allocator.

On each chain, the following contracts are deployed:

1. ALMProxy: Entity that holds funds and interacts with external contracts (e.g. DssAllocator, PSM). Thus, it holds the required privileges to interact with other contracts.
2. Controllers: Dictate which operations an ALMProxy shall perform. Note that multiple controllers could point to the same ALMProxy.
3. RateLimits: Computes limits for liquidity flows.

Please consult the Spark ALM Controller Reviews ([v1.7.0](#) and [v1.10.0 diff](#)) and Grove ALM Controller Review [v1.8.0](#) for more information.

2.2.2 PAU

The Laniakea Upgrade introduces PAU, a new architecture of the ALM Controller with improved efficiency and simplified management for automated capital deployment at scale.

PAU preserves the ALMProxy and RateLimits mechanism while replacing the Controller with a "Diamond Proxy" and deploying integrations as different contracts. This mitigates the bytecode size limit of the Legacy ALM Controller, simplifies the new integration addition and controller upgrades, and enables shared integrations across several Liquidity Layers.

Please consult the [Laniakea Docs](#) for more information.

2.2.3 Integrations

The following integrations are implemented and librarificated.

Freezer: in **both** controllers.

`removeRelayer()`: The *freezer* can revoke the RELAYER role from a given address, effectively removing a relayer from the controller.

Slippage Configuration: in **both** controllers.

`setMaxSlippage()`: The *admin* can configure maximum slippage (1e18 precision) per pool or token address. This shared configuration is used by multiple integrations: Aave, Curve, Uniswap V3, Uniswap V4, and OTC.

Arbitrary Token Transfer: integration in **both** controllers.

`transferAsset()`: ERC-20 assets can be transferred out manually using this function respecting the rate limit, including the destination address. Note that asset-destination pairs are rate limited. Further, the function is intended to be used for integration with other systems where direct token transfers are needed (e.g. Spark Vaults V2).

DSS Allocator: only in **mainnet** controller.

`mintUSDS()` / `burnUSDS()` leverage AllocatorVault to mint or burn (`draw()` or `wipe()`) USDS. Minting is rate limited while burning performs cancelling of the rate limit.

Dss LitePSM: only in **mainnet** controller.



swapUSDSToUSDC() / swapUSDCToUSDS() leverage the PSM to swap between USDS and USDC without fees (buyGemNoFee() and sellGemNoFee()) and are subject to rate limits where swapping back to USDS cancels the rate limit.

Spark PSM: only in **foreign** controller.

depositPSM() / withdrawPSM() enable depositing / withdrawing specific assets to / from Spark PSM with respect to separate rate limits.

Circle CCTP: in **both** controllers.

1. setCCTPMaxFeeCap(): restricted to the *admin*; sets a nominal maximum fee cap (cctpMaxFeeCap) for all CCTP-supported domains.
2. setMintRecipient(): restricted to the *admin*; sets the token recipient of a destination domain when bridging USDC with CCTP (Circle's Cross-Chain Transfer Protocol).
3. transferUSDCToCCTP(): leverages CCTP v2 to bridge USDC to a recipient (expected to be another ALMProxy) on a foreign domain which is subject to rate limits. Additionally, there is a rate limit for all CCTP transfers. The function has two overloads: one uses the default MAX_FEE and the other accepts a caller-specified maxFee. In both cases, the fee is validated against the cctpMaxFeeCap and must be strictly less than each chunked transfer amount.

ERC-4626: in **both** controllers.

depositERC4626() / withdrawERC4626() / redeemERC4626() enable integrations with ERC-4626 vaults. Deposits / withdrawals should respect the respective rate limit on each vault, the shares minted / assets withdrawn will be credited to the ALMProxy. The deposit rate limit will be replenished during a withdrawal or redemption, which is subject to the cap (maxAmount) of the limit and will revert if the limit is not configured (maxAmount==0). The *admin* can set maximum exchange rate thresholds using setMaxExchangeRate(), which takes a number of shares and the maximum expected assets for those shares. When depositing, the function validates that the actual exchange rate does not exceed the configured maximum, protecting against exchange rate manipulation attacks. ERC4626-related functions include slippage protection parameters: minSharesOut for deposits, maxSharesIn for withdrawals, and minAssetsOut for redemptions.

Aave: in **both** controllers.

depositAave() / withdrawAave() enable integrations with Aave V3 pools. Deposits / withdrawals should respect the respective rate limit on each aToken, and the newly minted aToken / underlying withdrawn will be credited to the ALMProxy. Note that both functions are separately rate limited per aToken. The Aave deposit rate limit will be replenished during a withdrawal, which is subject to the cap (maxAmount) of the limit and will revert if the limit is not configured (maxAmount==0). The *admin* must configure a maxSlippage value for the aToken. After supplying assets to the Aave pool, the function verifies that the aTokens received are at least amount * maxSlippage / 1e18, protecting against unfavorable exchange rates.

Ethena: only in **mainnet** controller.

1. setDelegatedSigner() / removeDelegatedSigner(): enables configuring any amount of delegated signers. The assigned delegated signers must explicitly accept the delegation with confirmDelegatedSigner() on Ethena before they can sign any Order on behalf of the ALMProxy. An Order is an expirable intent to mint or redeem USDe with fixed spending collateral and outcome USDe. In addition, it requires a minter and burner role on Ethena minting contract to submit the tx with the Order and signature prepared by the delegated signers.
2. prepareUSDeMint() / prepareUSDeBurn(): to mint or burn USDe, an approval can be triggered from the ALMProxy to the Ethena minting contract. Further operations from the delegated signers, Ethena minter and burners are required to complete the action.
3. Once the ALMProxy obtained USDe, it can be deposited into sUSDe to earn an interest. The deposit into sUSDe follows the typical ERC-4626 deposit interface.

4. `cooldownAssetsSUSDe()` / `cooldownSharesSUSDe()`: initiates a withdrawal of sUSDe that requires a cooldown period followed by an `unstake()` call since the instant withdrawal of sUSDe is currently blocked. These functions burn the shares and credit the USDe to the USDeSilo contract.
5. `unstakeSUSDe()`: enables the exit of USDe to the ALMProxy once the `cooldownDuration` is reached.

Note that minting, burning and the cooldown are separately rate limited.

Superstate Minting: only in **mainnet** controller.

`subscribeSuperstate()` allows minting USTB with USDC. The function is subject to rate limits and approvals are granted before calling the respective functions on Superstate.

Note that redemptions are not supported but USTB can be transferred if necessary.

Maple: only in **mainnet** controller.

1. Deposits happen through the existing ERC-4626 deposit function.
2. `requestMapleRedemption()` / `cancelMapleRedemption()`: withdrawals are not ERC-4626 compliant and work based on redemption requests, which can also be cancelled. It is assumed that manual withdrawal is disabled for ALMProxy on Maple, hence tokens will be pushed to ALMProxy once the redemption is processed. Should ALMProxy be configured to manual withdrawal on Maple, *relayers* have to trigger an explicit ERC-4626 `redeem()` on ALMProxy to retrieve the tokens, which further requires a withdrawal limit being configured for the maple pool.

Note that `requestMapleRedemption()` is rate limited per maple token while `cancelMapleRedemption()` only requires that the respective rate limit is configured (`maxAmount > 0`) but does not consume any rate limit.

Centrifuge: in **both** controllers.

1. `requestDepositERC7540()`: allows the relayer to initiate a call to `requestDeposit()` where assets are transferred to an escrow.
2. `claimDepositERC7540()`: allows to finalize the deposit and claim all the available shares once the deposit request is fulfilled.
3. `requestRedeemERC7540()`: initiates a withdrawal with shares transferred to escrow.
4. `claimRedeemERC7540()`: finalizes the redeem and retrieves the max withdrawable assets.
5. `cancelCentrifugeDepositRequest()` / `cancelCentrifugeRedeemRequest()`: allows to cancel an unfinalized deposit / redeem request.
6. `claimCentrifugeCancelDepositRequest()` / `claimCentrifugeCancelRedeemRequest()`: allows to finalize the cancel deposit / redeem and claim the available assets / shares once the request is fulfilled.
7. `transferSharesCentrifuge()`: allows transferring share class tokens to a cross-chain recipient address. The recipient for each destination chain (`centrifugeId`) must be explicitly configured by the *admin* using `setCentrifugeRecipient()`. The transfer amount is rate-limited based on the token and the destination chain.

Note that request operations are rate limited individually per token while other operations require that the respective rate limit exists.

Curve StableswapNG: in **both** controllers.

`swapCurve()` / `addLiquidityCurve()` / `removeLiquidityCurve()`: enables swaps and liquidity management respecting the defined slippage and rate limits.

DaiUsds: only in **mainnet** controller.

swapDAIToUSDS() / swapUSDSToDAI() support converting DAI to USDS and vice versa using the DAI-USDS converter without rate limits.

LayerZero: in **both** controllers.

1. `setLayerZeroRecipient()`: the *admin* can set the `LayerZeroRecipient` of each `destinationEndpointId`.
2. `transferTokenLayerZero()`: triggers a `send()` operation on an OFT to bridge the OFT supported token to a `destinationEndpointId`. Fees are estimated and attached and rate limit per OFT and `destinationEndpointId` is respected. The function is payable, hence the *relayers* or the controller itself (if it holds native token) pay the bridging fee. It supports OFTs that require approval prior to `send()`. Any excess native fee (`msg.value - fee.nativeFee`) is refunded back to the caller.

Farm: only in **mainnet** controller. Expected to be used with Sky's farms.

1. `depositToFarm()`: deposits the farm's staking token into the farm, limited by the rate limit configured for this farm.
2. `withdrawFromFarm()`: withdraws the staked token which at the same time claims the rewards for the ALMProxy. The withdrawal is also rate limited per farm.

Spark Vaults V2: in **both** controllers.

`takeFromSparkVault()` enables taking assets from SparkVault. The function is rate limited per SparkVault. Note that funds can be returned with `transferAsset()`.

Pendle: in **both** controllers.

The *admin* must configure the Pendle router address using `setPendleRouter()` before redemptions can be executed.

`redeemPendlePT()`: Allows the controller to redeem matured Principal Tokens from expired Pendle markets. The function requires the market to have reached expiry before redemption can occur. The redemption converts PT tokens back to their underlying yield-bearing token through Pendle's router, with the amount being rate-limited per market. The function automatically calculates the minimum expected output based on the Yield Token's `pyIndexCurrent()`, applying a small buffer to account for potential rounding errors. A relayer-specified slippage protection ensures redemptions execute as expected. Rate limit is decreased according to the `totalTokenOutAmount`. The redeemed assets are returned to the ALMProxy.

Uniswap V3: in **both** controllers.

1. `swapUniswapV3()`: Allows the relayer to execute token swaps through whitelisted Uniswap V3 pools. The swap is rate-limited per input token and pool combination. Price movement and slippage are constrained by the admin (the Governance) through `swapMaxTickDelta` and `maxSlippage`. A TWAP oracle is used to determine the price limit tick.
2. `addLiquidityUniswapV3()`: Allows the relayer to mint new Uniswap V3 positions or add liquidity to existing positions. The tick range for new positions must fall within admin configured bounds (`addLiquidityTickBounds`) for this pool. For existing positions, the provided tick range must be within the position's existing tick range. The deposited amounts for both tokens are rate-limited per token and pool. A TWAP-based validation ensures minimum amounts are reasonable given the fair market price and the `maxSlippage` parameter.
3. `removeLiquidityUniswapV3()`: Allows the relayer to decrease liquidity from existing Uniswap V3 positions belonging to the ALMProxy and collect the resulting tokens. The withdrawn amounts are rate-limited per token and pool. The function collects all owed tokens (including any accumulated fees) to the ALMProxy. Slippage is checked against the amounts returned by `decreaseLiquidity()` directly, rather than comparing proxy balance differences. Additionally, `maxSlippage` must be configured (non-zero) for the pool.

The *admin* (the governance) can configure pool parameters using `setUniswapV3PoolMaxTickDelta()`, `setUniswapV3AddLiquidityLowerTickBound()`, `setUniswapV3AddLiquidityUpperTickBound()`, and `setUniswapV3TWAPSecondsAgo()`. Additionally, the *admin* must set the swap router and position manager addresses using `setUniswapV3SwapRouter()` and `setUniswapV3PositionManager()`. Such a configuration effectively whitelists the pool for use by the relayer.

Merk1: in **both** controllers.

`toggleOperatorMerk1()`: Allows the relayer to toggle an operator address for the Merkl distributor enabling or disabling that operator's ability to claim rewards on behalf of the ALMProxy. The operator can only trigger claims, funds are always sent to the user, the ALMProxy. On both controllers, the *admin* must configure the distributor address using `setMerk1Distributor()`.

WrapProxyETH: only in **mainnet** controller.

`wrapAllProxyETH()`: Wraps proxy's native Ether into WETH.

Etherfi (weETH): only in **mainnet** controller.

It enables the ALMProxy to deposit into and withdraw from EtherFi's liquidity pool. The integration consists of two components:

WEETHLib: A library providing the following functions:

1. `deposit()`: Converts WETH to weETH. The function unwraps WETH, deposits ETH into EtherFi's liquidity pool to receive eETH shares, then wraps the eETH into weETH. A `minSharesOut` parameter provides slippage protection.
2. `requestWithdraw()`: Initiates a withdrawal from weETH by unwrapping weETH to eETH, then requesting a withdrawal from the EtherFi liquidity pool. An NFT receipt is minted to the WEETHModule contract representing the withdrawal request. A `minEETHShares` parameter is defined for weETH unwrapping.
3. `claimWithdrawal()`: Claims a finalized withdrawal request, receiving ETH which is then wrapped to WETH and transferred to the ALMProxy.

WEETHModule: An upgradeable UUPS proxy contract that:

- Implements `onERC721Received()` and acts as the recipient for withdrawal request NFTs.
- Provides `claimWithdrawal()` function that claims ETH from finalized requests, wraps to WETH, and transfers to ALMProxy.
- Restricted so only the ALMProxy can call `claimWithdrawal()`.
- Only `DEFAULT_ADMIN_ROLE` can upgrade the implementation.

Rate limits apply to deposit and request withdrawal. Claim withdrawal requires that the request withdrawal rate limit is configured (`maxAmount > 0`) but does not consume any rate limit.

MainnetController Functions:

- `depositToWeETH(uint256 amount, uint256 minSharesOut)`: Deposits WETH to weETH with slippage protection.
- `requestWithdrawFromWeETH(address weETHModule, uint256 weETHShares, uint256 minEETHShares)`: Initiates withdrawal request.
- `claimWithdrawalFromWeETH(address weETHModule, uint256 requestId)`: Claims finalized withdrawal.

The following integrations are assumed to be correct and secure since they are not audited by ChainSecurity:

UniswapV4: only in **mainnet** controller.

The *admin* can configure tick limits for pools using `setUniswapV4TickLimits()`.

1. `swapUniswapV4()`: Swaps an input token to an output token using a UniswapV4 pool. `maxSlippage` is used for slippage protection.
2. `mintPositionUniswapV4()`: Mints a new position in a UniswapV4 pool. The position must fall into the admin defined tick limits.
3. `increaseLiquidityUniswapV4()`: Adds liquidity to an existing position, respecting the admin defined tick limits.
4. `decreaseLiquidityUniswapV4()`: Removes liquidity from an existing position.

WSTETH: only in **mainnet** controller.

1. `depositToWstETH()`: Deposits WETH to Lido and gets WSTETH.
2. `requestWithdrawFromWstETH()`: Requests a withdrawal from WSTETH.
3. `claimWithdrawalFromWstETH()`: Claims a finalized request.

OTCSwap: only in **mainnet** controller.

The *admin* can configure OTC exchanges using `setOTCBuffer()`, `setOTCRechargeRate()`, and `setOTCWhitelistedAsset()`.

1. `otcSend()`: Starts an OTC swap given a whitelisted exchange and asset when the swap is ready. Transfers asset out to the exchange directly.
2. `otcClaim()`: Claims the output token of an OTC swap from the `otcBuffer`.
3. `getOtcClaimWithRecharge()`: Returns the current claimed OTC swap amount and the recharge amount, representing the slippage tolerance over time.
4. `isOtcSwapReady()`: Checks if the current amount claimed and slippage tolerance exceed the `maxSlippage` of this exchange.

2.2.4 Upgrade Phases

Phase1: Integration Merge & Librarification

In this phase (**Version 1**), functionalities of [Spark ALM Controller](#) and [Grove ALM Controller](#) are merged and refactored into dedicated libraries:

1. Some integrations are already librarificated at v1.10.0: Aave, Curve, ERC4626, LayerZero, PSM, UniswapV4, and WEETH.
2. CCTPV2 is borrowed from Grove ALM Controller and used in PAU instead of CCTPV1.
3. ApproveLib is a helper library shared among other libraries.
4. Other existing integrations of Spark ALM Controller are further refactored into libraries: WETH, WSTETH, PSM3, Ethena, Maple, SuperState, DAIUSDS, SPK Farm, Spark Savings Vault, OTC Swap and ERC20 Transfer.
5. Grove-specific integrations are added as new libraries: Centrifuge, ERC7540, Merkl, Pendle, UniswapV3.

Most importantly, the following stay the same in this upgrade. Hence no change is needed for the existing integrations:

1. The rate limit prefixes and keys.
2. The way the rate limit is consumed or increased.
3. The existing integration configurations and logic.

Note: Some integrations only exist on Mainnet and others only exist on L2s. For each chain, the ALM Controller now can support all these integrations while certain integrations are disabled with unconfigured contract addresses or unconfigured rate limits.

In addition, some integrations may not be used by other SubDAOs due to the requirement of other contracts, for instance SparkVaultLib.

2.3 Trust Model

ALMProxy: The *admin* is fully trusted, otherwise, it can set up controllers and trigger any calls with the privilege of ALMProxy. The *controller* is also trusted.

In addition, the ALMProxy requires several roles to operate, which are assumed to be set up properly by governance, for instance:

1. It requires bud role to swap without fee on DSS LitePSM.
2. It requires wards role on AllocatorVault to draw() and wipe() USDS.
3. It needs sufficient allowance from AllocatorBuffer to move minted USDS.

MainnetController and ForeignController:

1. The *admin* is fully trusted, otherwise they can DoS the controller, or steal the bridged money on the destination domain by changing the mint recipient.
2. The *relayer* is semi-trusted, and they can only change the liquidity allocation in the worst case. The *freezer* is also semi-trusted which can temporarily DoS the controller in the worst case.

Before initializing the contracts, the governance should always carefully examine whether the deployed contracts match the expectations.

RateLimits: The *admin* is fully trusted to configure the limit data and *controller* correctly.

Sky ecosystem contracts (e.g. AllocatorVault, AllocatorBuffer, DAI-USDS converter, Spark PSM, Sky farms) are assumed to be the expected, correctly deployed contracts and are trusted to function as documented.

The following 3rd-party integration requires an extended trust model:

- It is assumed Diamond PAU will not interact with weird ERC-20 (rebasings / low decimals / ...) and ERC-4626 vault (low token decimals / without share inflation protection / ...). Otherwise, for instance, in case an ERC-4626 has low decimals, a *relayer* may amplify the loss due to rounding errors in shares conversion with many calls for ALMProxy on L2s.
- Diamond PAU is subject to the inherent risks of these protocols (i.e. risks of upgradeability, RWAs, governance ...) and generally the third party protocols receiving funds are assumed to be non-malicious.

Arbitrary ERC-4626: The *admin* should only configure a deposit rate limit for trusted and correct external ERC-4626 vaults.

Aave and Aave-like protocols: Aave governance can adjust the parameters for the reserves such as setting it inactive, paused or frozen or upgrading the pool / aToken implementations. Diamond PAU is subject to the potential risks of Aave, and Aave governance is fully trusted to not misbehave. Similarly, note that other Aave-like protocols such as Spark are trusted in the same way if support for the respective LP tokens is added.

Ethena: The funds deposited to Ethena are subject to the risks of depegging and minting / burning limits of Ethena. The minter and burner of USDe are fully trusted, otherwise, they can DoS the USDe minting and burning. The delegated signers assigned by *relayers* are semi-trusted:

1. There could be a race condition if multiple delegated signers exist, who can sign orders with different volume and quotes.
2. The signers may not sign any order or not use all the allowance prepared by the ALMProxy.
3. The signers can sign with bad quotes regardless of the ones Ethena returned. It is assumed **the fully trusted Ethena minter / redeemer will never submit these malicious orders**. (see [A Compromised Ethena Minter Or Redeemer May Execute A Bad Order](#))

4. Holding sUSDe further implies the risks of blacklisting (for deposit / withdraw / transfer) and confiscating, the admins and other privileged roles of sUSDe are trusted to not misbehave.
5. The *relayers* are semi-trusted, a compromised [Relayer Can DoS SUSDE Unstaking](#), which requires *admin* privilege to remove the malicious *relayer*. In addition, as a *relayer* can assign any delegated signer, a compromised *relayer* can incur all the aforementioned risks of a malicious delegated signer.

Superstate USTB: The owner of USTB is fully trusted, otherwise it can for instance:

1. Block the deposit / withdrawals by pausing USTB.
2. Manipulate the conversion price by changing the oracle. It is assumed the owner of AllowList is trusted and will whitelist the ALMProxy. USTB and AllowList contracts are deployed behind upgradeable proxies, hence the proxy admins are trusted to not upgrade to malicious implementations.

Maple: Maple governance is fully trusted which oversees the protocol, who can upgrade the implementations of the manager contracts which feature important logic and accounting. The redeemer of MapleWithdrawalManager is trusted to process the redemptions in time. The governor and the pool delegator are trusted since they can trigger the update of unrealized loss accounting on the LoanManager contract, which will influence the redemption conversion rate. The protocol admin and pool delegator are also trusted, otherwise they may interfere with the automatic redemption by setting the manual withdrawal for the ALMProxy.

Centrifuge: *Centrifuge is assumed to be the only ERC-7540 integrated.* The privileged roles (wards) are fully trusted, in particular:

- The wards of the ERC-7540 vaults are fully trusted, otherwise they can DoS the system by changing the manager or asyncRedeemManager contract.
- The wards of the asyncRedeemManager are fully trusted, otherwise they may 1) stop fulfilling deposits, redemptions, or cancellation requests; 2) fulfill the requests with a bad conversion rate and incur a loss to the users.

Further, shares transfer related infrastructures (Gateway, Spoke, Hub...) and the privileged roles are expected to work correctly and honestly. Otherwise, the shares in flight may be stuck due to censorship.

Curve StableswapNG pools:

1. The tokens in the pool are pegged. Otherwise, swaps (or adding imbalanced liquidity) to the depegged tokens are allowed by the controller, hence incur a loss to the ALM Proxy.
2. The *admin* will set up the slippages properly (`setMaxSlippage()`). Otherwise, curve-related functions could be DoSed if it is >100%, or significant slippage could be allowed if it is too small.

LayerZero:

LayerZero is out of scope for this review, it is assumed to work as documented, and trusted to behave correctly, and deliver messages to the correct destination:

1. The OFT and recipient of each `destinationEndpointId` configured are fully trusted, otherwise, it can steal the funds being transferred, or upgrade and bridge unexpected tokens. The configuration required for managing the OFT on multiple chains is considered out of scope and assumed to be correct.
2. The LayerZero executor is trusted to always deliver messages with the correct provided options. However in case LayerZero behaves incorrectly, for example due to a compromise, an L2 finality issue, or any other failure the rate limit functionality caps the amount of funds at risk.

CCTP v2: Circle's burn and mint system and its attestation service are trusted to process transfers correctly.

Pendle: Pendle contracts (Router, markets, and SY, PT, YT tokens) are trusted to function correctly. The owner of the router is fully trusted, while PT and YT contracts have no owner. The SY contract implementation and any privileged roles on it are fully trusted. For more details, see [Risk of Pendle Integration](#). The controller handles redemptions only; PT tokens arrive at the ALM proxy separately (e.g., transferred after having been purchased through a brokerage service). This may be done with funds of the ALMProxy and a deposit using `transferAsset()`.

Uniswap V3: Uniswap V3 pool contracts and the position manager are trusted to function correctly. The controller relies on pool state (current tick, token addresses, fee tier) for swap execution and liquidity operations. The Relayer can only execute operations on pools that have been configured by the admin (the governance). We expect that only legitimate pool addresses are whitelisted. The TWAP oracle functionality relies on the pool's observation cardinality being sufficient for the configured `twapSecondsAgo` period.

Merk1: The Merkl distributor contract is trusted. On both controllers, the *admin* must correctly configure the distributor address. Operators toggled via `toggleOperatorMerkl` gain the ability to claim rewards on behalf of the ALMProxy.

SparkVault: The SparkVault is expected to implement the `take` function and transfer the requested assets to the ALMProxy.

WEETHModule: Its *admin* (`DEFAULT_ADMIN_ROLE`) is fully trusted and can upgrade the implementation. In the worst case it can steal the funds in the pending withdraw request NFTs it owns.

weETH (EtherFi): EtherFi protocol is assumed to work as documented. EtherFi governance is trusted to not misbehave. They can DoS the system with pausing or other admin privileges. In the worst case, they can upgrade the EtherFi contracts to steal the funds.

UniswapV4: Only the hookless 1:1 stablecoin pool is assumed to be used and the tick limits are assumed to be properly configured and adjusted.

WSTETH: The Lido governance is assumed to be honest since it can upgrade the related contracts. The Lido AccountingOracle is assumed to report honestly.

Upgradeable OTCBuffer: The *admin* is fully trusted and can upgrade the implementation. In the worst case it can steal funds held by the buffer.

2.3.1 Changelog

In **Version 3**, the following changes were introduced:

- Added `cctpMaxFeeCap` state variable and `setCCTPMaxFeeCap()` admin function to both controllers. A new `transferUSDCToCCTP()` overload accepts a caller-specified `maxFee`, validated against the cap and each chunk size.
- `LayerZeroLib` refunds excess native fee (`msg.value - fee.nativeFee`) back to the caller after `send()`.
- `CentrifugeLib` calls `baseManager()` instead of `manager()` to match the updated Centrifuge vault interface.
- `UniswapV3Lib.removeLiquidity()` checks slippage against `decreaseLiquidity()` return values directly instead of comparing proxy balance differences. A `maxSlippages[pool] != 0` check is enforced before removing liquidity.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the [Resolved Findings](#) section. The findings are split into these different categories:

- **Security**: Related to vulnerabilities that could be exploited by malicious actors
- **Design**: Architectural shortcomings and design inefficiencies
- **Correctness**: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	8

- [Centrifuge Deposit / Redemption Can Be DoSed by Cancellation](#) **Risk Accepted**
- [Centrifuge Tranche Token Price May Change Between Request Submission and Execution](#) **Risk Accepted**
- [Lack of Rate Limits](#) **Code Partially Corrected** **Acknowledged**
- [LayerZero Dangling Approvals](#) **Code Partially Corrected** **Acknowledged**
- [Maple Redemption Can Be DoSed](#) **Risk Accepted**
- [Over-reduced Limit in Maple Redemption](#) **Risk Accepted**
- [Relayer Can DoS SUSDE Unstaking](#) **Risk Accepted**
- [Revoking Unused Approval](#) **Acknowledged**

5.1 Centrifuge Deposit / Redemption Can Be DoSed by Cancellation

Security **Low** **Version 1** **Risk Accepted**

CS-SKYDPAU-001

The *relayers* can request to cancel pending deposits / redemptions on Centrifuge and claim them later once fulfilled. Note that in case there is a pending deposit cancellation, no new deposit can be made (same for redemption). Consequently, a compromised *relayer* can DoS new deposit requests by triggering a deposit cancellation (same for redemption) until the existing pending deposit is cancelled or fulfilled.

Risk accepted:

Sky is aware of the risk.

5.2 Centrifuge Tranche Token Price May Change Between Request Submission and Execution

Correctness **Low** **Version 1** **Risk Accepted**

CS-SKYDPAU-002

When requesting a redemption from a Centrifuge ERC-7540 vault, the rate limit is decreased by an estimation of the withdrawable assets (`convertToAssets(shares)`) based on the latest tranche token price.

However, the tranche token price may change between the redemption request submission and execution, hence the actual withdrawable assets after execution may not match the rate limit decreased at submission time.

5.3 Lack of Rate Limits

Design **Low** **Version 1** **Code Partially Corrected** **Acknowledged**

CS-SKYDPAU-003

The LayerZero integration implements rate limits per OFT and destination pair. In contrast, the CCTP integration implements a limit per destination and a global CCTP limit (always in USDC). The LayerZero integration however lacks a notion of global limits per token and, hence, an inconsistency between the bridging rate limits for CCTP and LayerZero exists.

Further, the LayerZero integration might pay fees in native tokens. However, the outbound amount of native tokens is not rate limited. Thus, a *relayer* colluding with an endpoint operator could drain the native token balance.

Acknowledged:

Sky has acknowledged the inconsistency between the bridging rate limits for CCTP and LayerZero.

5.4 LayerZero Dangling Approvals

Correctness **Low** **Version 1** **Code Partially Corrected** **Acknowledged**

CS-SKYDPAU-004

Pending approvals might remain. Namely, the `OFTReceipt` contains the `amountSentLD` and the `amountReceivedLD` amounts where `amountSentLD` corresponds to the amount actually debited from the user. Hence, `send()` could potentially pull less tokens than approved (e.g. LayerZero dust removal) and pending approvals could exist. Optimally, the approvals should be revoked to prevent dangling approvals.

Note that pending approvals might introduce a corner case if ZRO are held and bridged. Namely, a dangling approval could allow for a `lzTokenFee > 0` to be collected for ZRO OFTs.

Acknowledged:

Sky has acknowledged the dangling approvals could still exist.

5.5 Maple Redemption Can Be DoSed

Security Low Version 1 Risk Accepted

CS-SKYDPAU-005

Maple redemption can be DoSed by a compromised *relayer* in two ways:

1. Each user can have at most 1 redemption request in `MapleWithdrawalManager`. Hence a compromised *relayer* can keep triggering dust redemptions and block the legitimate redemptions from honest *relayers*. In this case, the honest *relayers* have to cancel the dust redemptions first before triggering a legitimate one.
2. Requesting a maple redemption will consume rate limit, whereas cancelling a redemption will not recharge the limit. Consequently, if the whole rate limit is consumed by a compromised *relayer*, other *relayers* will not be able to trigger future redemptions.

Risk accepted:

Sky has added a test (`Attacks.t.sol, test_attack_compromisedRelayer_delayRequestMapleRedemption`) to demonstrate the mitigation: if a malicious *relayer* delays redemption, the *freezer* can remove the compromised *relayer* and revert to the governance *relayer*. This prevents the compromised *relayer* from continuing the attack, allowing the governance *relayer* to cancel and submit the legitimate request.

5.6 Over-reduced Limit in Maple Redemption

Correctness Low Version 1 Risk Accepted

CS-SKYDPAU-006

In `requestMapleRedemption()`, the redemption limit will be reduced given the conversion rate between the shares and the assets with `convertToAssets()`.

In `MaplePool`, function `convertToAssets` assumes the pool holds `totalAsset()` without unrealized loss. However, when the redemption is processed with `processRedemptions()`, the withdrawable amount takes the unrealized loss into consideration.

Consequently, there would be a discrepancy between the rate limit decrease and the actual received tokens in the event of unrealized loss.

5.7 Relayer Can DoS SUSDE Unstaking

Security Low Version 1 Risk Accepted

CS-SKYDPAU-007

In Ethena, two steps are required to convert sUSDe to USDe:

- A cooldown must be initiated first, which (1) burns the shares and credits the USDe to the USDeSilo contract (2) resets the `cooldownEnd` to be `cooldownDuration` from `currentBlock.timestamp`. Note the step (2) will extend any existing cooldown asset to another `cooldownDuration`.
- When the `cooldownEnd` is reached, the sUSDe can be unstaked and the USDe will be credited to a specified receiver.

Consequently, a malicious relayer can keep triggering new cooldowns with as little as 1 wei asset to block previous exits from sUSDe to USDe, hence DoS the sUSDe to USDe conversion.

Note: Sky was aware of this issue and had reported to us before the audit. In addition, in case a malicious *relayer* DoSed the sUSDe `unstake()`, Sky will freeze the controller, remove the malicious *relayer*, and reactivate the controller again.

In version 11, since function `freeze()` has been removed, instead of freezing the whole controller, the *freezer* should revoke the *RELAYER* role from the malicious *relayer*.

5.8 Revoking Unused Approval

Design

Low

Version 1

Acknowledged

CS-SKYDPAU-008

The *freezer* can remove *relayers* from the `MainnetController` which prevents *relayers* from triggering any more interactions or funds transfers from the `ALMProxy`.

However, since the Ethena integration requires actions from several external parties (see [Allowance For Ethena Minter May Not Be Consumed](#)), the actual transfers of underlying assets to mint or redeem USDe may happen even after the *relayer* is disabled.

Acknowledged:

Sky acknowledged the issue and decided not to change the code since Ethena is fully trusted.

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the [Open Findings](#) section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	0
Informational Findings	2

- [Typos](#) **Code Corrected**
- [msg.value Validation in transferTokenLayerZero](#) **Code Corrected**

6.1 Typos

Informational **Version 1** **Code Corrected**

CS-SKYDPAU-012

1. In WSTETHLib: IERC20ike.
2. In USDSLlib: ApprobeLib.
3. In MainnetController: cancelation.

Code corrected:

All the typos have been corrected.

6.2 msg.value Validation in transferTokenLayerZero

Informational **Version 1** **Code Corrected**

CS-SKYDPAU-021

The relayer attaches msg.value to calls to transferTokenLayerZero() to pay for the LayerZero V2 fees. Note that there might be two scenarios:

- The relayer does not provide sufficient value (e.g. pricing changed between transaction sending and arrival). Then, if the controller does not hold the relevant native token delta, the call reverts. However, that works as expected.
- Similarly, the relayer might provide a msg.value that is too high due to similar reasons. In such cases the native token could be stuck in the controller.

Ultimately, the second scenario could lead to native tokens in the controller. Typically, they will not be used. However, technically the relayer could reuse them for future LayerZero V2 fees. However, refunding the relayer with remaining delta might be more meaningful.

Code corrected:

Since **Version 3** the excessive Ether will be refunded back to the message sender.

7 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

7.1 Allowance For Ethena Minter May Not Be Consumed

Informational **Version 1** **Acknowledged**

CS-SKYDPAU-013

The integration with Ethena minter for USDe minting and burning requires external parties' (delegated signers, Ethena minter and redeemer) actions. The *relayer* can only trigger the `approve()` from the ALMPProxy and expect the consecutive actions will be completed by the external parties.

In the following cases the allowance may not be fully consumed:

- The delegated signers sign orders with smaller volume which do not consume all the allowance.
- The Ethena minter or redeemer refuse to submit the order, which blocks the minting or redeeming and does not consume the allowance.
- The expected minting and burning may not be executed successfully due to the restrictions on Ethena minter such as the volume exceeds per block limit.

Consequently, the actual amount used in the interactions may be less than the amount tracked by the rate limit.

Acknowledged:

Sky acknowledged the issue and decided not to change the code.

7.2 Can Only Collect Uniswap V3 Position Fees When Removing Liquidity

Informational **Version 1** **Acknowledged**

CS-SKYDPAU-014

The `removeLiquidity` function in `UniswapV3Lib.sol` will internally call the `NonfungiblePositionManager`'s `collect` function to collect fees from the position. The `removeLiquidity` function does not allow being called when removing zero liquidity from the position. There exists no separate `collect()` function in `UniswapV3Lib.sol` that allows a relayer to only collect the fees of a given position. As a result, the only way to collect the fees from the position is to remove a non-zero amount of liquidity by calling the `removeLiquidity()` function.

Acknowledged:

Sky acknowledged the issue and decided not to implement a fix.

7.3 Code Redundancy

Informational

Version 1

Code Partially Corrected

Acknowledged

CS-SKYDPAU-009

1. ApproveLib is imported but never directly used in MainnetController.
2. Duplicated functions: RateLimitHelpers library block duplicates the free functions.
3. Duplicated Interfaces: some interfaces are defined multiple times across different contracts (e.g. IERC20Like, IERC4626Like).

Acknowledged:

1. ApproveLib import has been removed.

Others are acknowledged by Sky.

7.4 CurveLib: Rounding Error Amplification

Informational

Version 1

Acknowledged

CS-SKYDPAU-010

The CurveLib now executes the following equivalent computation to get the MinimumAmountOut given the configured maxSlippage:

```
// Makes the assumption that value in should equal value out.
uint256 valueIn = amountIn * rates[inputIndex] / 1e18;
uint256 equivalentAmountOut = valueIn * 1e18 / rates[outputIndex];
uint256 MinimumAmountOut = equivalentAmountOut * maxSlippages[pool] / 1e18;

require(
    minAmountOut >= MinimumAmountOut,
    "CurveLib/min-amount-not-met"
);
```

It performs three divisions, and the rounding error of the first two division results is amplified by the consecutive multiplications. Consequently, the MinimumAmountOut could be underestimated.

Acknowledged:

Sky responded:

We opted to make the code easier to understand here, acknowledge small rounding error got introduced.

7.5 Dust eETH Shares When Wrapping Into weETH

Informational

Version 1

Acknowledged

CS-SKYDPAU-015



In WEETHLib, when wrapping eETH into weETH, the following steps are performed:

1. Convert eETH shares to an eETH amount using `liquidityPool.amountForShare()`, which performs the following share to underlying computation that rounds down.

```
eETHAmount = (_share * getTotalPooledEther()) / totalShares;
```

1. Call `WEETH.wrap()` with `eETHAmount` from the previous step, which computes the shares (rounding down) to be transferred as follows:

```
shares = (_amount * eETH.totalShares()) / totalPooledEther;
```

Consequently, not all the eETH shares may be wrapped due to rounding errors in the conversion. Consider the following examples:

```
# Example 1
Pool State:
  totalShares:      10000000000000000007
  totalPooledEther: 10000000000000000003
  ExchangeRate:     ~1.0
User State:
  userShares:       10000000000000000000
  userBalanceEther: 9999999999999999999
  sharesToTransfer: 9999999999999999999
  dust:             1

# Example 2
Pool State:
  totalShares:      8161165065522594326093698
  totalPooledEther: 4469658072974448790601728
  ExchangeRate:     0.547674
User State:
  userShares:       76540196863890419613696
  userBalanceEther: 41919077248541685964672
  sharesToTransfer: 76540196863890419613694
  dust:             2
```

7.6 Inconsistency

Informational **Version 1** **Acknowledged**

CS-SKYDPAU-011

1. Sanity Checks: some integration-related addresses (`pendleRouter`, `merkleDistributor`, `uniswapV3PositionManager`, `uniswapV3Router`) are not configured in the constructor hence staying 0 before they are configured. It is not checked in the `UniswapV3Lib` that the respective addresses (`uniswapV3Router`, `uniswapV3PositionManager`) are non-zero before the logic execution.
2. Function Arguments: Named argument syntax is used in some integrations but not others (positional arguments), leading to inconsistent readability and robustness across different integrations.
3. ApproveLib: Most libraries embed the `ApproveLib` for token approval logic, while some libraries use customized approval logic (i.e. `PSMLib`, `CCTPLib` and `WSTETHLib`).

4. RateLimit Invocation: In some Libs, rateLimit operation is inlined, while in others it is wrapped into an internal function.
5. External Function librarification: Not all external functions are librarified, in particular:
 1. Some configuration setters are not librarified (e.g. `setPendleRouter()`, `setMintRecipient()`).
 2. The access control related functionalities are not librarified. (e.g. `grantRole()`, `revokeRole()`, `removeRelayer()`).
 3. Auto generated Getters are not librarified.
 4. Events of some integrations are refactored to libraries while others stay in the controller (e.g. `PendleRouterSet`).
6. Interfaces: Most contracts define minimal interfaces to use, while PendleLib, TransferAssetLib and MapleLib import OZ interfaces.
7. Constants Naming: operation keys are defined as constants in respective libraries. Most constant names are defined without integration name, while some constant names still contain the integration name (e.g. `LIMIT_USDE_BURN`).
8. Address Passing: Most libraries accept the target integration contract as argument, while some libraries hardcode the integration contract addresses (e.g. `WEETHLib`, `UniswapV4Lib`).

7.7 Maple Manual Withdraw May Be Enabled

Informational

Version 1

Risk Accepted

CS-SKYDPAU-016

A Maple redemption requires two steps:

1. The user submits a redemption request.
2. The privileged redeemer processes the request.

In a typical path, no more user interactions are required after step 1, and the underlying tokens will be automatically sent to the user in step 2.

However, in case manual withdrawal is enabled for the user, another call to `MaplePool.redeem()` must be initiated to fulfill the withdrawal and trigger the underlying token transfer.

Note that manual withdrawals can only be enabled by the privileged roles (pool delegator and protocol admins) of `MapleWithdrawalManager` with `setManualWithdrawal()`. In this case, the ALM Proxy has to explicitly call `redeem` (ALM Controller's `redeemERC4626()`) to finalize the redemption. And this requires a `LIMIT_4626_WITHDRAW` configured on the ALM Controller for this `MaplePool`. In addition, the manually withdrawable shares will be internally accounted in the `MapleWithdrawalManager`, hence the share balance of `ALMProxy` (`balanceOf()`) will not contain this.

7.8 Missing Validation of Pendle Market V3

Informational

Version 1

Acknowledged

CS-SKYDPAU-017

Pendle is designed to be permissionless. Anyone can create an SY token and deploy PT, YT and a market for the tokens using the factory. In `PendleLib`, the rate limit checks validate that the Pendle market is a contract whitelisted by the *default admin*, but do not verify that it is a legitimate contract deployed by the `PendleMarketFactoryV3` using `isValidMarket()`. Note that market validation can also be performed when configuring the rate limits.

Acknowledged:

Sky states:

The check can be done in spell tests that are onboarding a particular market. On top of that onboarding of new assets is very thoroughly reviewed by multiple parties anyway.

7.9 Per Pool maxSlippage Instead of per Operation

Informational **Version 1** **Acknowledged**

CS-SKYDPAU-018

A single maxSlippage value is defined per pool (by the admin/governance) and is used for all operations of a given pool. This could be considered to be a design issue, since it might be necessary to have a different max slippage tolerance for a removeLiquidity() operation (e.g. due to impermanent loss) than for a swap() or addLiquidity() operation. Being able to define a different maxSlippage value per operation would allow for more flexibility and control over the slippage tolerance on a pool.

Acknowledged:

Sky acknowledged the issue and decided not to implement a fix since they will mostly be operating in stablecoin-stablecoin pools, and therefore the slippage considerations should be fairly similar.

7.10 Use UniversalRouter Instead of SwapRouter02

Informational **Version 1** **Acknowledged**

CS-SKYDPAU-019

Sky indicated they are using Uniswap's SwapRouter02 instead of UniversalRouter since the latter does not return the amountOut.

```
uint256 startingBalance = IERC20(cache.tokenOut).balanceOf(address(context.proxy));
uint256 amountOut = _callSwap(context, params, cache);
uint256 endingBalance = IERC20(cache.tokenOut).balanceOf(address(context.proxy));
```

In the swap function the returned amountOut is used to update the rate limit. However, the before and after balance delta of the tokenOut is calculated to check the validity of the relayer-provided minAmountOut value. This delta indicates the actual amountOut and therefore using this value instead of the SwapRouter02 returned value is more accurate. This would allow using the modern, and Uniswap-recommended, UniversalRouter instead of the much older SwapRouter02.

Acknowledged:

Sky acknowledged this and decided not to change the code.

7.11 Withdraw From Aave Can Be Blocked By LTV=0 Asset

Informational Version 1 Acknowledged

CS-SKYDPAU-020

When an asset is deposited under a user for the first time, the asset will be automatically configured as collateral if its LTV is non-zero and it is not in isolation mode.

In case a user has an asset enabled as collateral which has $LTV == 0$, the user will not be able to withdraw any other assets that have $LTV > 0$.

As a consequence, the following theoretical attack is possible:

- An attacker observed an asset that has $LTV > 0$ is going to be configured to $LTV == 0$ on Aave.
- It can supply on behalf of the ALMProxy (or send directly) a dust amount of this aToken.
- After the parameter change on Aave, the asset has $LTV == 0$. The attacker successfully DoS the ALMProxy, which will not be able to withdraw the desired aToken (i.e. aUSDS, aUSDC...) from Aave.

Note that there is no rate limit and token restriction on function `withdrawAave()`. The *relayer* can withdraw the full balance of the asset with $LTV == 0$, which resets the `usingAsCollateral` flag to `false` and recovers the ALMProxy from the DoS.

Acknowledged:

Sky has acknowledged this issue and stated a rate limit for the $LTV = 0$ asset will be added to withdraw this asset in case this attack happens.

7.12 setOTCRechargeRate May Flip OTC Swap Status

Informational Version 1 Acknowledged

CS-SKYDPAU-022

`setOTCRechargeRate()` changes the recharge rate without checking if there is an ongoing OTC swap. When called during an active swap, the new rate is applied retroactively to calculate the swap's readiness status, which can cause `isOtcSwapReady()` to flip immediately.

This occurs because `getOtcClaimWithRecharge()` uses the current `rechargeRate18` for the entire elapsed time since `sentTimestamp`:

```
otc.claimed18 + (block.timestamp - otc.sentTimestamp) * otc.rechargeRate18;
```

8 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

8.1 A Compromised Ethena Minter Or Redeemer May Execute A Bad Order

Note Version 1

Ethena's minter and redeemer are two crucial roles that can submit the signed orders to the Ethena minting contract. Since the delegated signers are only semi-trusted and can be malicious, the minter and redeemer are fully trusted to never submit bad orders signed by the malicious delegated signers (also described in [Trust Model](#)).

In the worst case, if Ethena's minter or redeemer are compromised, they may collude with a malicious delegated signer to execute an order with a bad quote that drains the approved USDC from ALMProxy.

8.2 ABI Changes

Note Version 1

Offchain programs / services should be aware of the abi changes (compared to the initial ALM controller):

- Mainnet Controller now has functionalities and events of new integrations.
- Following events are removed from Mainnet Controller's abi: LayerZeroRecipientSet, MaxExchangeRateSet, MintRecipientSet, UniswapV4TickLimitsSet, and OTC related events.
- Nonce field has been removed from event CCTPTransferInitiated.
- Events OTCTBufferSet and OTCRechargeRateSet do not contain the old values any more.

8.3 Aave Interprets Uint256 Max Withdrawal as Full Withdrawal

Note Version 1

The *relayers* should be aware that Aave will interpret a withdrawal with `type(uint256).max` amount as a full withdrawal of the user's balance. The *relayers* should be careful of this special behavior if they are dependent on the input amount.

8.4 Asynchronous Operations May Be Interfered With

Note Version 1

The execution of some asynchronous operations may be interfered with and unable to finalize due to another operation. For instance, the operation of `prepareUSDeMint()` will be initiated to mint USDe,

which simply grants allowance of tokens to be deposited. Before the 3rd-party operation to consume the allowance, another operation, i.e. `swapCurve()`, may use up the tokens. This may cause the 3rd-party operation to fail due to insufficient token balances.

The *relayers* should be careful of the asynchronous operations and avoid the interference of different operations.

8.5 Centrifuge Shares Transfer Refund to ALMProxy

Note **Version 1**

When transferring Centrifuge shares to another chain with `transferSharesCentrifuge()`, the *relayer* has to attach `msg.value` to pay for the transfer fuel. Note that in case the fuel is excessive, it will be refunded to the ALMProxy instead of the *relayer*.

8.6 Curve Withdrawal Slippage

Note **Version 1**

When removing liquidity from Curve with `removeLiquidityCurve()` a balanced withdrawal is performed. Note that the performed slippage protection is not strictly necessary. Namely, assuming tokens are pegged, that is due to no negative slippage in terms of "underlying value" being possible.

As a consequence, note that the *relayer* will typically be forced to simulate the transaction to be able to provide rough values suitable to pass the check.

8.7 Inconsistent Swap Rate Limit Decrease for Curve

Note **Version 1**

When adding liquidity to Curve, a swap can occur. Note that the rate limit adjustment is inconsistent with the adjustment in the swap function. Consider the following example:

1. Assume that swapping 50 token A returns 49 token B.
2. When using the swap function, the rate limit is reduced by 50.
3. Assume that when adding liquidity with 100 token A and 0 token B, the internal swap swaps so that 50 token A and 49 token B are added.
4. The swap performed is effectively the swap from 1.
5. However, the rate limit adjustment will be the average of the input and output deltas and will thus be 49.5.

Ultimately, there can be swap rate limit discrepancies between swap and adding liquidity. However, note that this is intended according to Sky.

8.8 Maple Deposit Ignores Unrealized Losses

Note **Version 1**



When depositing into the MaplePool, shares are minted assuming there are no unrealized losses from the underlying loan managers, hence this deposit will bear part of the impairment immediately. In addition, withdrawals from MaplePool will bear existing unrealized loss and forfeit the potential recovery of the impairment. The *relayers* should monitor Maple's loan and unrealized loss status before deposits and withdrawals to avoid loss to the ALMProxy.

8.9 OFT Considerations

Note Version 1

Governance should be aware that certain OFTs are not supported. Below is a list of considerations to make when adding support for an OFT:

- OFTs that try to pull more than was specified are not supported due to lack of sufficient approval.
- OFTs that could try to burn (without approval) more than was specified are generally not supported. If more would be burned than specified, the rate limit accounting could be incorrect. Thus, such OFTs should not be added.
- OFTs with inherent rate limits could lead to unsuccessful operations. More specifically, the executions could revert due to OFT rate limits.
- OFTs should be ensured to follow the OFT standard correctly. Additionally, the underlying token should not be allowed to change or similar as this could lead to rate limit violations.
- The gas cost for configured `destinationEndpointId` should be carefully monitored to ensure that the hardcoded value of `200_000` is sufficient.

8.10 OFTs With Mandatory Fee in IzToken Are Not Supported

Note Version 1

Function `transferTokenLayerZero()` queries the fees with `quoteOFT()` prior to `send()` to prepare the fee payment by attaching required native tokens. Even though it quotes OFT with flag `_payInLzToken=false`, it does not guarantee the fee contains 0 IzToken. Hence, in case part of the fee must be paid in IzToken, `send()` will fail due to insufficient approval of IzToken.

In summary, OFTs that always require part of the fee in IzToken are not supported by Diamond PAU hence should not be used.

8.11 Risk of Pendle Integration

Note Version 1

The following risks are introduced by the Pendle integration:

1. The Pendle [router proxy contract](#) forwards calls to different implementations (facets). It is controlled by its owner and upgradeable. Consequently, all funds allocated to the Pendle integration could be lost if the owner was malicious.
2. The SY token contract can have various implementations. Consequently, if it was malicious, the funds for redemption could be lost.
3. The redemption relies on the `yt.pyIndexCurrent()` computation and an honest relayer submitting a `minAmountOut`. There are no admin-defined slippages.

Markets and tokens should be properly inspected before being added to the rate limits.

8.12 Special Cases Handling

Note Version 1

The ALM's functionality can be extended by allowing new controllers. Some currently unresolvable scenarios could be resolved in the future if needed. For example:

1. Assume it is desired that for an L2, all funds are bridged back to mainnet. However, in case the PSM3 never holds sufficient USDC to bridge back to L1, funds will remain on L2. As a result, another controller could be whitelisted that initiates redeeming the PSM shares against the other two assets to then bridge them back to mainnet through the respective bridges.
2. The mint recipient for CCTP could be blacklisted. That effectively could DoS the USDC bridging. In that case, a new controller could be added that allows calling CCTP's `replaceDepositForBurn` to resolve the issue.

Ultimately, some unlikely (and intentionally unhandled) issues may arise with the existing controllers. To resolve such issues, new controllers can be added.